

CASE STUDIES ON WINDOWS, LINUX AND UNIX OPERATING SYSTEM

Case Study 1: Windows Operating System

Introduction

Windows is a family of operating systems developed by Microsoft. It is primarily designed for personal computers, laptops, and enterprise systems. The modern versions (Windows 10, 11, and Windows Server editions) are based on the Windows NT architecture. It provides a user-friendly graphical interface along with powerful background processing capabilities.

Architecture

The Windows architecture is divided into the following layers:

1. **Hardware Abstraction Layer (HAL):**

Acts as an interface between system hardware and the OS kernel, providing hardware independence.

2. **Kernel:**

The core component managing processor scheduling, interrupt handling, and memory management.

3. **Executive Services:**

Manages process control, I/O, memory, security, and inter-process communication.

4. **Device Drivers:**

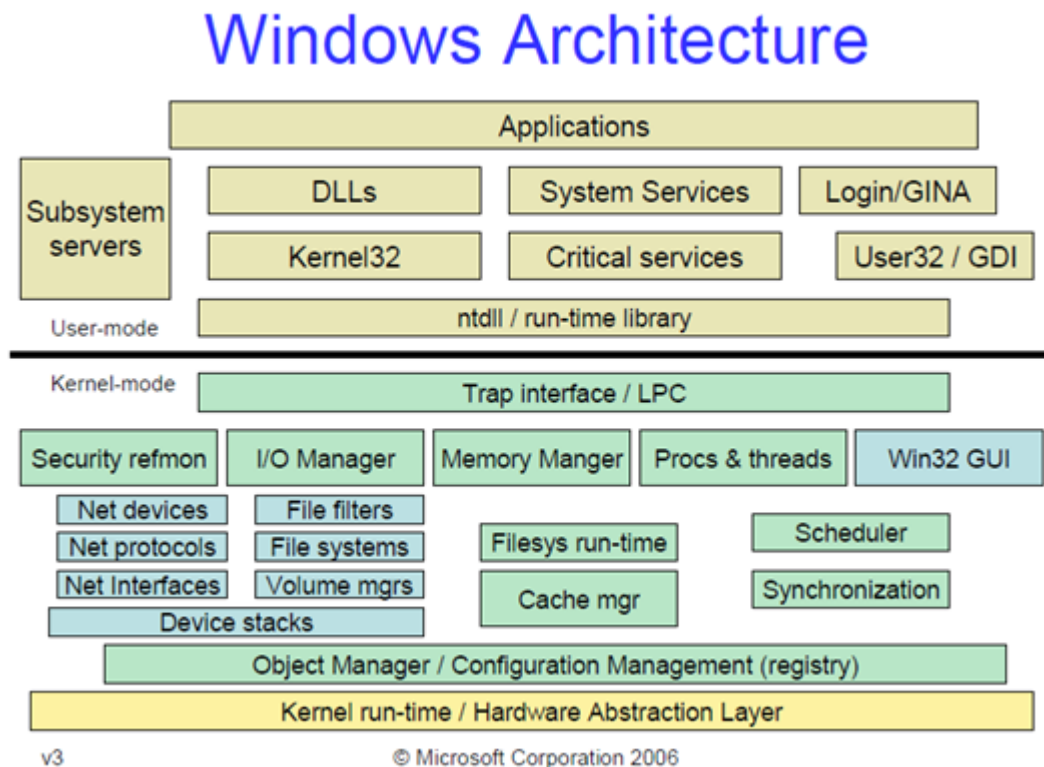
Allow communication between hardware devices and the operating system.

5. **System Processes and Subsystems:**

Includes the Win32 subsystem (main user interface) and system services like csrss.exe and smss.exe.

6. User Interface:

Provides GUI-based interaction through components like taskbar, start menu, and desktop.



Key Features

- Preemptive multitasking and multithreading
- Graphical user interface with rich interaction
- Security through access tokens, user authentication, and UAC (User Account Control)
- NTFS file system with journaling and encryption
- Virtual memory and paging
- Registry for centralized configuration management

System Calls in Windows

System calls in Windows are not directly invoked by the user. They are implemented as **Win32 API calls** that interact with the kernel. Some important examples include:

Function	Purpose
CreateProcess()	Creates a new process
ReadFile()	Reads data from a file or device
WriteFile()	Writes data to a file or device
CreateFile()	Opens or creates a file
VirtualAlloc()	Allocates virtual memory
ExitProcess()	Terminates a process
WaitForSingleObject()	Waits for a process or thread to finish execution

Analysis: Windows OS

Windows uses a hybrid kernel that combines the performance of a monolithic kernel with modular features. Its multitasking is based on a **priority-driven, preemptive scheduling algorithm**. Each process is assigned a priority level, and the CPU executes the highest priority process first.

Memory management is handled through **paging and demand paging**. When physical memory runs low, pages are swapped to disk, providing the illusion of more memory (virtual memory). The file system NTFS supports features like compression, encryption, and recovery logs, enhancing reliability.

Security is a major focus in Windows. It supports access control lists (ACLs), user authentication, and permissions for files and processes. Networking capabilities are provided through TCP/IP stack integration.

Short Questions

1. Explain the architecture of the Windows operating system.
2. What is the function of the Hardware Abstraction Layer?
3. Describe the role of the Windows kernel.
4. Explain process creation using CreateProcess() system call.
5. How does Windows manage virtual memory?

Case Scenario

A university computer lab uses Windows 11 systems for student projects. Students often run large applications like MATLAB, Python IDEs, and browsers simultaneously. Over time, the systems slow down.

Analysis:

- Virtual memory is overused due to limited RAM.
- Paging increases disk I/O, leading to thrashing.
- Background services consume resources.

Solution:

- Increase physical memory.
- Disable unnecessary startup services.
- Adjust process priorities using Task Manager.

Case Scenario Questions:

1. Identify the reasons behind performance degradation in the above scenario.
2. Suggest memory management improvements for better performance.
3. How can the Task Manager be used to monitor and control resource usage?

Case Study 2: Linux Operating System

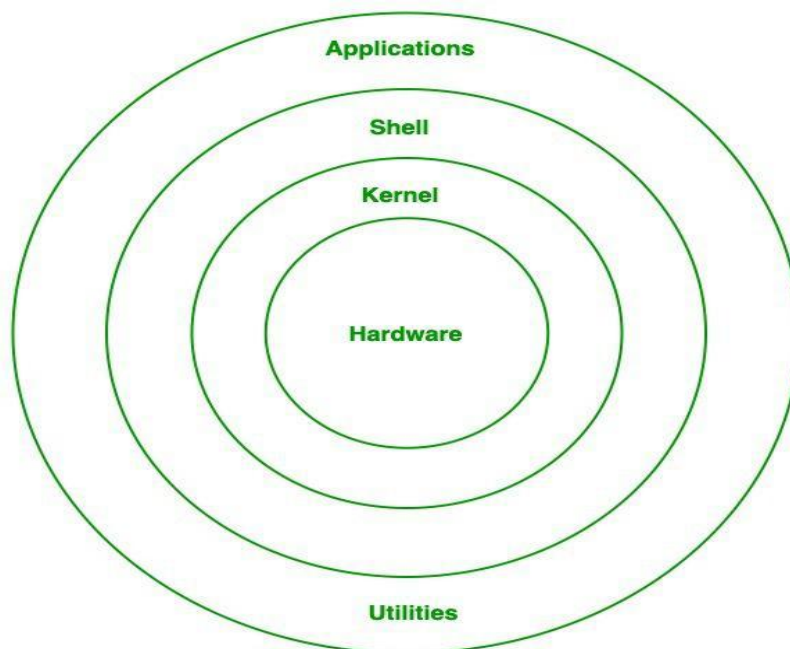
Introduction

Linux is an open-source operating system modeled on UNIX. It uses a monolithic kernel and is widely used in servers, desktops, and embedded systems. Linux provides both command-line and graphical interfaces, making it versatile for developers and administrators.

Architecture

Linux architecture consists of the following components:

1. **Hardware:** Physical devices like CPU, memory, and I/O peripherals.
2. **Kernel:** The core part that manages CPU scheduling, memory, devices, and system calls.
3. **System Libraries:** Provide standard functions for user applications.
4. **System Utilities:** Manage system operations such as file handling and process control.
5. **User Interface:** Command-line shells (bash, zsh) and graphical interfaces (GNOME, KDE).



Key Features

- Multiuser and multitasking environment
- Open-source and customizable
- Efficient process management using fork() and exec()
- Supports multiple file systems (ext4, XFS, Btrfs)
- Shell scripting and automation capabilities
- Strong security through file permissions and user privileges

System Calls in Linux

System calls act as an interface between user programs and the kernel. Some common Linux system calls are:

System Call	Purpose
fork()	Creates a new process
exec()	Executes a new program
read()	Reads data from a file descriptor
write()	Writes data to a file descriptor
open()	Opens a file
close()	Closes a file
wait()	Waits for a child process
exit()	Terminates a process
kill()	Sends a signal to a process

Analysis: Linux OS

Linux uses a **monolithic kernel**, which means all essential components run in kernel mode. It supports loadable kernel modules that can be added or removed without rebooting. Process management is highly efficient using the fork() and

exec() system calls. The fork() system call creates a child process, while exec() replaces it with a new program.

Memory management uses both **paging** and **segmentation**, and the system allocates memory dynamically through demand paging. The Linux scheduler implements **Completely Fair Scheduling (CFS)**, ensuring fair CPU time distribution.

Linux provides a robust permission system. Each file or directory has permissions for the owner, group, and others, which control read, write, and execute access.

Short Questions

1. Explain Linux architecture in detail.
2. Describe the functions of fork() and exec() system calls.
3. How does Linux implement multitasking?
4. What are the advantages of the Linux file permission system?
5. What is the role of shell scripting in Linux?

Case Scenario

A start-up company deploys a web server using Ubuntu Linux to host its online applications. During peak hours, the website slows down, and the server becomes unresponsive.

Analysis:

- Multiple Apache processes consume high CPU.
- Cached memory usage increases with concurrent requests.
- Inefficient load balancing affects response time.

Solution:

- Configure swap space for memory overflow.
- Use monitoring tools like top, htop, and vmstat.
- Implement load balancing using Nginx or HAProxy.

Case Scenario Questions:

1. What system monitoring tools can help identify performance issues?

2. How can load balancing improve server performance?
3. Explain how Linux manages memory during high load conditions.

Case Study 3: UNIX Operating System

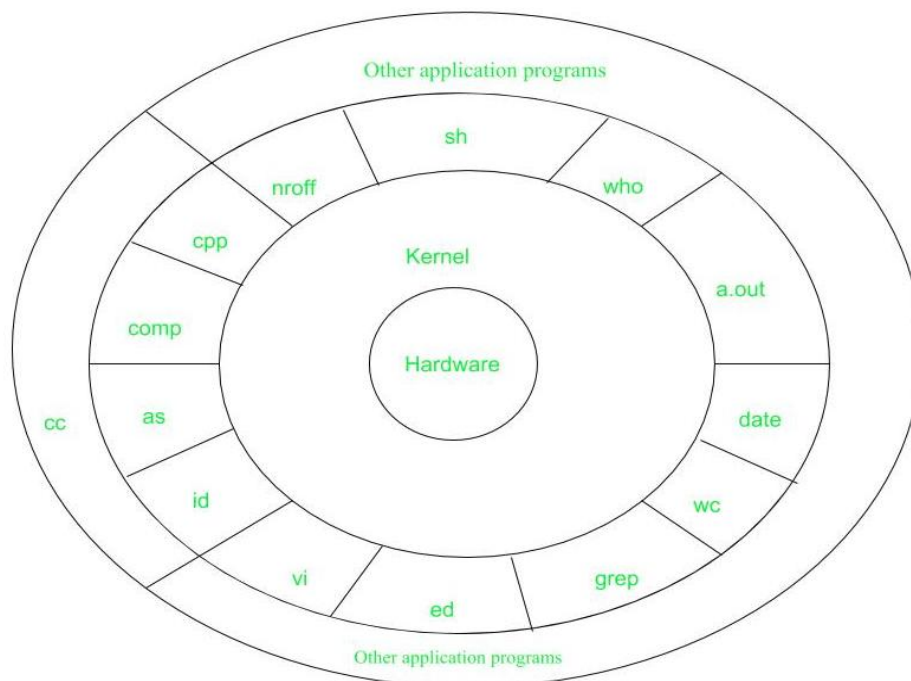
Introduction

UNIX is a multiuser, multitasking operating system developed at AT&T Bell Labs in 1969. It became the foundation for modern OS like Linux, Solaris, and macOS. UNIX emphasizes simplicity, security, and portability.

Architecture

The UNIX architecture has following components:

- Layer-1: Hardware: It consists of all hardware related information.
- Layer-2: Kernel: This is the core of the [Operating System](#). It is a software that acts as the interface between the hardware and the software. Most of the tasks like memory management, file management, network management, process management, etc., are done by the kernel.
- Layer-3: Shell commands: This is the interface between the user and the kernel. Shell is the utility that processes your requests. When you type in a command at the terminal, the shell interprets the command and calls the program that you want. There are various commands like [cp](#), [mv](#), [cat](#), [grep](#), [id](#), [wc](#), [nroff](#), [a.out](#) and more.
- Layer-4: Application Layer: It is the outermost layer that executes the given external applications.



Key Features

- Multiuser and multitasking
- Device independence
- Strong file and process security
- Hierarchical file system
- Command-line interface and shell scripting
- Portability and scalability

System Calls in UNIX

UNIX system calls are similar to Linux, as Linux evolved from UNIX. Some examples include:

System Call	Purpose
fork()	Creates a child process
exec()	Executes a new program
wait()	Waits for process termination
open()	Opens a file
read()	Reads from a file
write()	Writes to a file
close()	Closes a file
kill()	Sends signals to processes
chmod()	Changes file permissions

Analysis: Unix OS

UNIX uses a **monolithic kernel** with layered design. It supports multitasking using time-sharing and priority scheduling. Each process is uniquely identified by a process ID (PID). UNIX manages memory using segmentation and paging.

The file system is organized in a hierarchical structure, with / as the root directory. All devices, including hardware components, are represented as files, providing simplicity and consistency.

Security in UNIX is achieved through file permissions and user authentication. System administrators can control access using `chmod`, `chown`, and user group configurations.

Short Questions

1. Explain the layered architecture of UNIX.
2. Describe process management and scheduling in UNIX.
3. How does UNIX treat hardware devices?
4. Discuss the significance of file permissions in UNIX.
5. Compare UNIX and Linux systems.

Case Scenario

A research institution uses UNIX-based Solaris systems for data analysis. Multiple users execute computationally heavy programs, leading to delays and performance bottlenecks.

Analysis:

- Several processes run simultaneously, increasing CPU load.
- Limited physical memory leads to excessive swapping.
- Disk quotas are exceeded by some users.

Solution:

- Use `nice` and `renice` to adjust process priorities.
- Implement user disk quotas.

- Schedule batch jobs during low-usage periods.

Case Scenario Questions:

1. How can UNIX administrators manage process priorities?
2. What role do disk quotas play in performance management?
3. Suggest methods to balance workload among multiple users.

Comparative Summary

Feature	Windows	Linux	UNIX
Type	Proprietary	Open Source	Proprietary/Open
Kernel	Hybrid	Monolithic	Monolithic
User Interface	GUI-based	CLI + GUI	CLI-based
Security	Moderate (UAC)	Strong (Permissions)	Strong (Permissions)
File System	NTFS	ext4, XFS	UFS
Preferred Use	Personal, Business	Servers, Developers	Research, Enterprise